



吉林大学
JILIN UNIVERSITY



AVC Lab
Affective Vision Computing Lab

具身推理: Long-horizon Task Planning & Agent Memory 科研基础知识报告

Presenter: Jiaqi Shen
Date: 2026/9/5



01

Outline

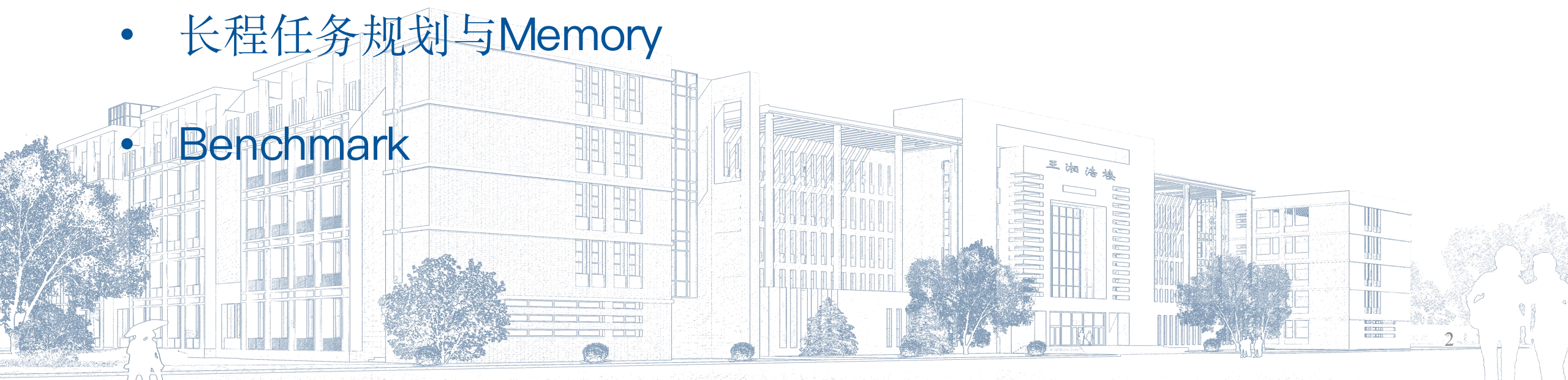
目录



AVC Lab

Affective Vision Computing Lab

- 具身推理
- 长程任务规划
- Agent Memory
- 长程任务规划与Memory
- Benchmark



02

具身推理

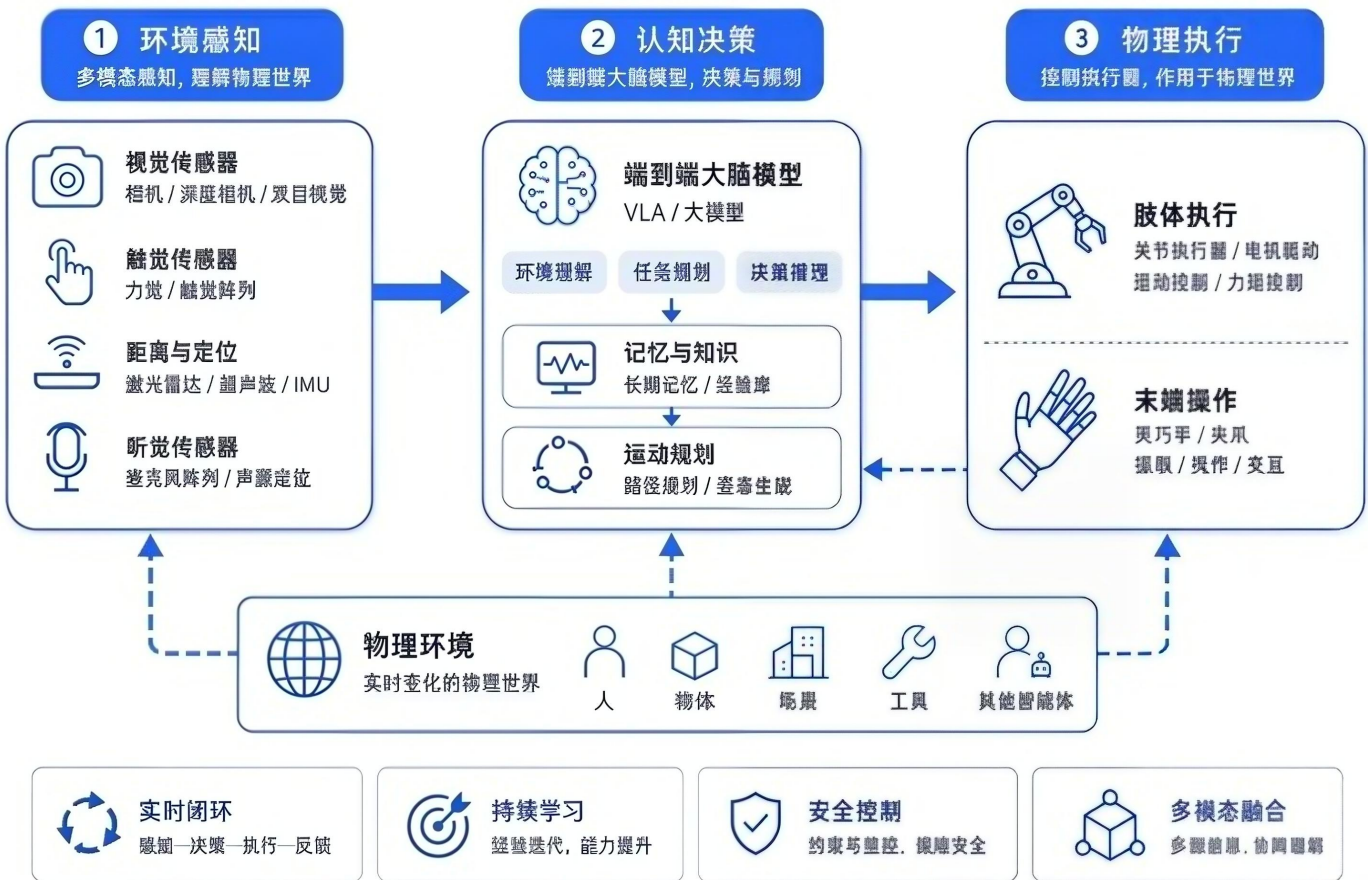
具身推理关注agent如何在世界中理解并行动



AVC Lab

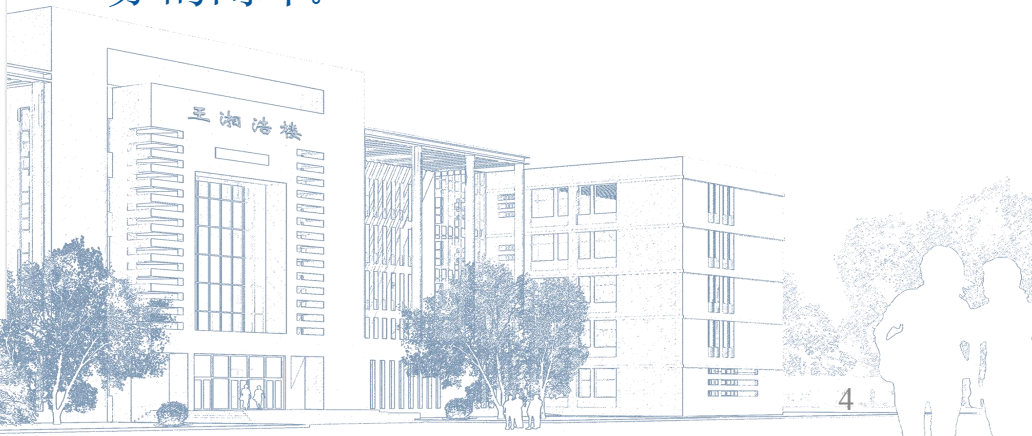
Affective Vision Computing Lab

具身智能是具备物理实体、能与环境交互并完成任务的智能系统，通过“感知—思考—行动”的闭环实现自主作业。



具身智能 (Embodied AI)

- 是人工智能的一个核心范式, 指能够在物理世界中自主感知、推理、决策并执行物理行动的智能系统。
- 与依赖数据输入输出的传统虚拟AI不同, 具身智能强调“感知—思考—行动”的闭环。

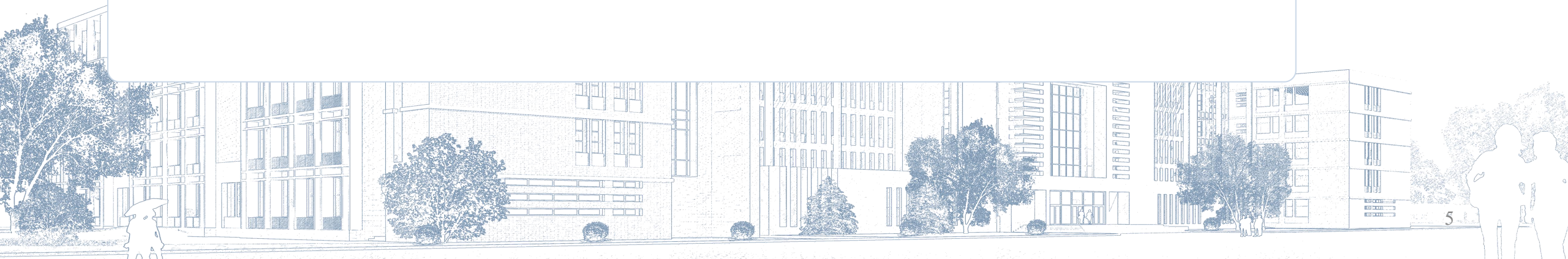




Embodied Reasoning

具身推理指 agent 将 perception、language、world state 和 action 约束联合起来，在可交互环境中进行目标理解、状态判断、动作选择和结果修正的过程。

- 推理对象不只是文本命题，还包括 objects、spatial relations、affordances 与动态反馈。
- 输出不只是答案，而是可在环境中执行的 action 或 action sequence。
- 正确性既取决于语义合理性，也取决于物理可行性与执行结果。



03

长程任务规划

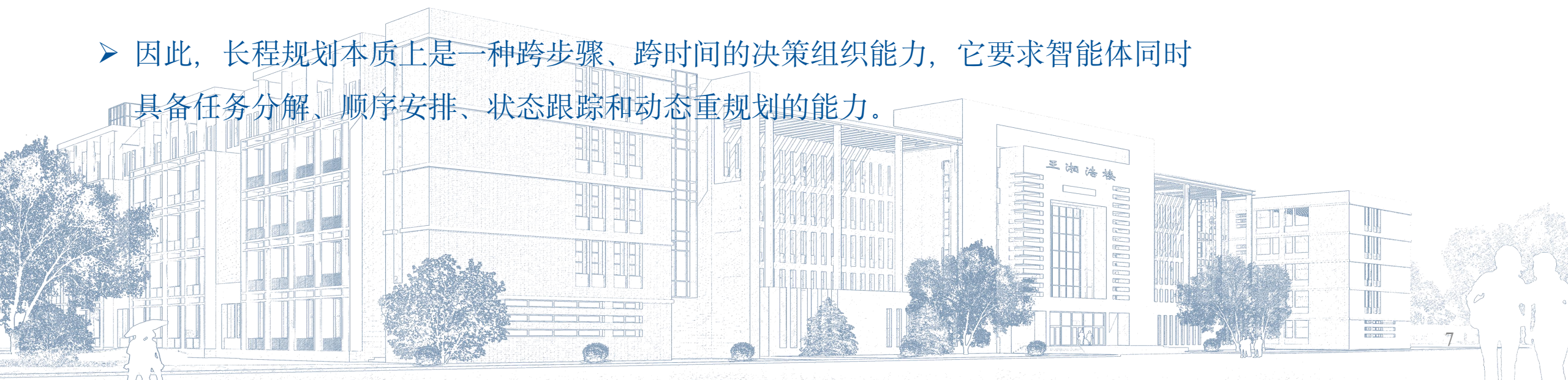
什么是长程任务规划？



AVC Lab

Affective Vision Computing Lab

- 长程任务规划是指智能体面对一个复杂目标时，不能依赖单一步骤完成，而必须先
将总体目标分解为多个子目标，再按照合理顺序逐步执行。
- 它的关键不只是“列出步骤”，更是要在执行过程中维护任务状态，判断当前子任务是
否完成，并在环境变化或执行失败时及时调整计划。
- 因此，长程规划本质上是一种跨步骤、跨时间的决策组织能力，它要求智能体同时
具备任务分解、顺序安排、状态跟踪和动态重规划的能力。



PLAN-AND-ACT ——先规划后动手



AVC Lab

Affective Vision Computing Lab

PLAN-AND-ACT: Improving Planning of Agents for Long-Horizon Tasks, ICML 2025

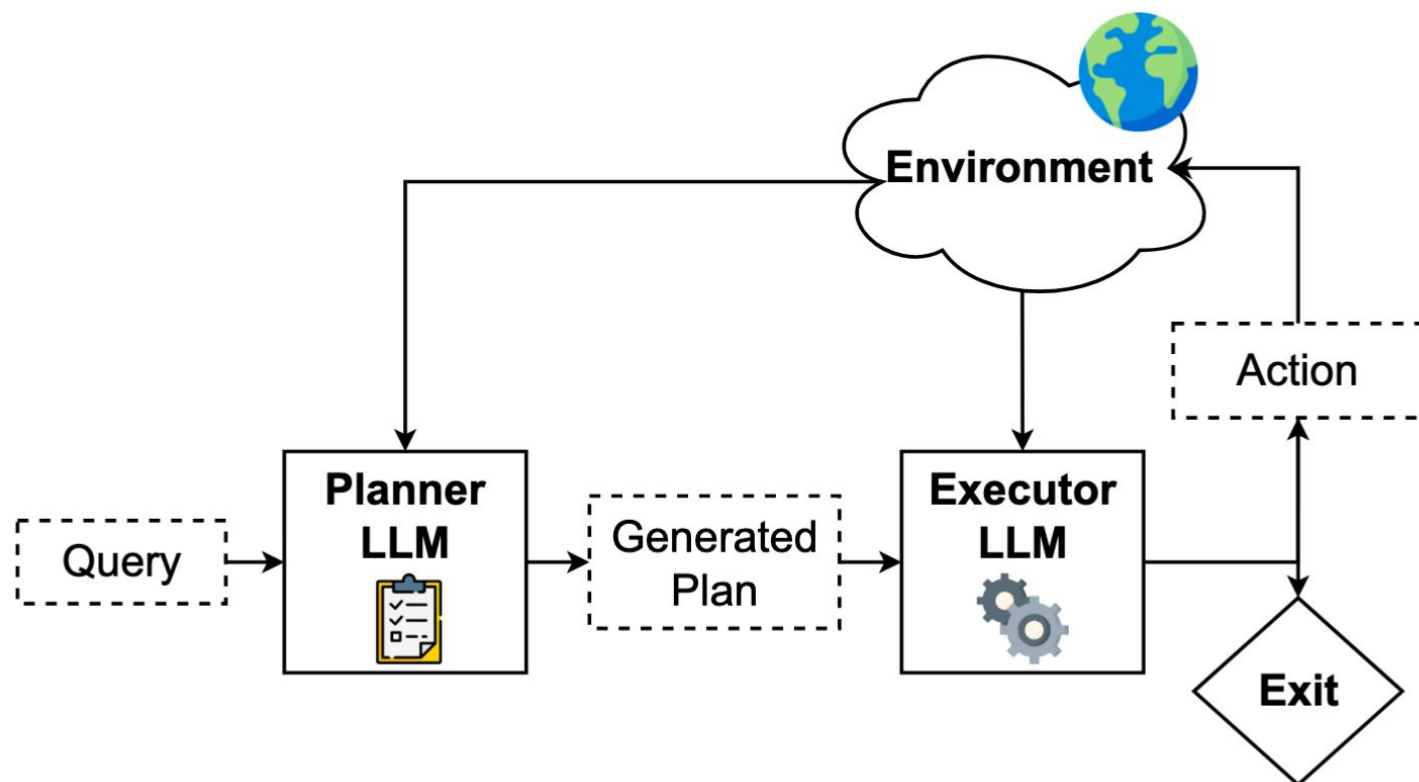
PLAN-AND-ACT 把系统切成两个专门模块。它给出了一套可扩展地训练 PLANNER 的合成数据方法，运行时结构只有 2 个 Agent。

➤ 1、PLANNER: 把目标拆成结构化计划

PLANNER 拿到用户查询后，输出一份结构化、高层级的计划

➤ 2、EXECUTOR: 把计划翻译成具体动作

➤ 3、动态重规划: EXECUTOR 每执行一步，PLANNER 就根据"当前状态 + 之前的计划 + 已做的动作"重新生成一份计划。



短时程任务 VS 长时程任务

维度	Short-horizon Task	Long-horizon Task
目标结构	单一目标或少量局部约束	目标可分解为多级 subgoals, 存在依赖顺序
反馈	很快看到 success / failure	delayed feedback, 错误可能很晚才暴露
状态管理	上下文窗口通常足够	需要显式 state tracking、memory 与 progress monitoring
规划形式	一步或少量步骤即可完成	需要 hierarchical planning、replanning、contingency handling
失败恢复	重新回答或重试即可	必须定位失败源, 并决定回滚、替换步骤还是重规划





长时程任务是具身推理最典型的压力测试

为什么二者天然相关

- 具身环境要求 agent 在多个观察-动作循环中持续推进目标。
- 每一步行动都会改变下一步的状态空间，规划错误会随时间放大。
- 长任务包含空间导航、对象操作、工具使用、人类反馈和失败恢复。
- 因此长时程任务把 perception、reasoning、planning、memory 同时推到前台。

典型场景

- 家庭机器人：找到杯子、清洗、放回指定位置。
- Web / OS agent：跨页面检索、填写表单、检查结果。
- Minecraft / open-world agent：采集资源、合成工具、探索未知区域。
- 实验室机器人：按 protocol 执行多步操作并记录异常。

当任务跨越很多步，agent 需要的不只是更强的模型，而是稳定的任务状态管理。

➤ 目标漂移与误差累积 (Goal drift and compounding error)

在上千步的执行中，智能体可能逐渐偏离最初目标；而且由于没有哪一步是完全可靠的，局部错误会沿轨迹不断累积，最终把系统带入错误的推理路径

➤ 上下文退化与上下文容量压力 (Context rot and context-limit pressure)

随着工作上下文被填满，智能体往往会在超过某个上下文利用阈值后突然退化；反过来，一个感知到上下文上限的智能体也可能过早终止，并把部分进展错误地宣告为成功

➤ 稀疏且延迟的奖励，以及不可逆性 (Sparse, delayed rewards and irreversibility)

许多长时程任务只在最后才返回奖励，这使智能体在中间决策上得到极其稀疏的学习信号；同时，智能体运行得越久，就越可能做出高风险且不可逆的动作，因此必须重视执行安全性与可恢复性。





ReAct 将 reasoning trace 与 action trace 交替生成

ReAct: Synergizing Reasoning and Acting in Language Models

ReAct = Reasoning (推理) + Acting (行动)

ReAct 的经典结构由三个部分组成: Thought、Action 和 Observation

1. Thought: AI用文字写下它当前的状态和下一步计划。
2. Action: AI执行一个具体的操作, 通常是调用某个工具。
3. Observation: 外部工具或环境返回的真实结果

这个三步循环会不断重复, 直到AI认为已经找到答案, 执行Finish行动结束任务。



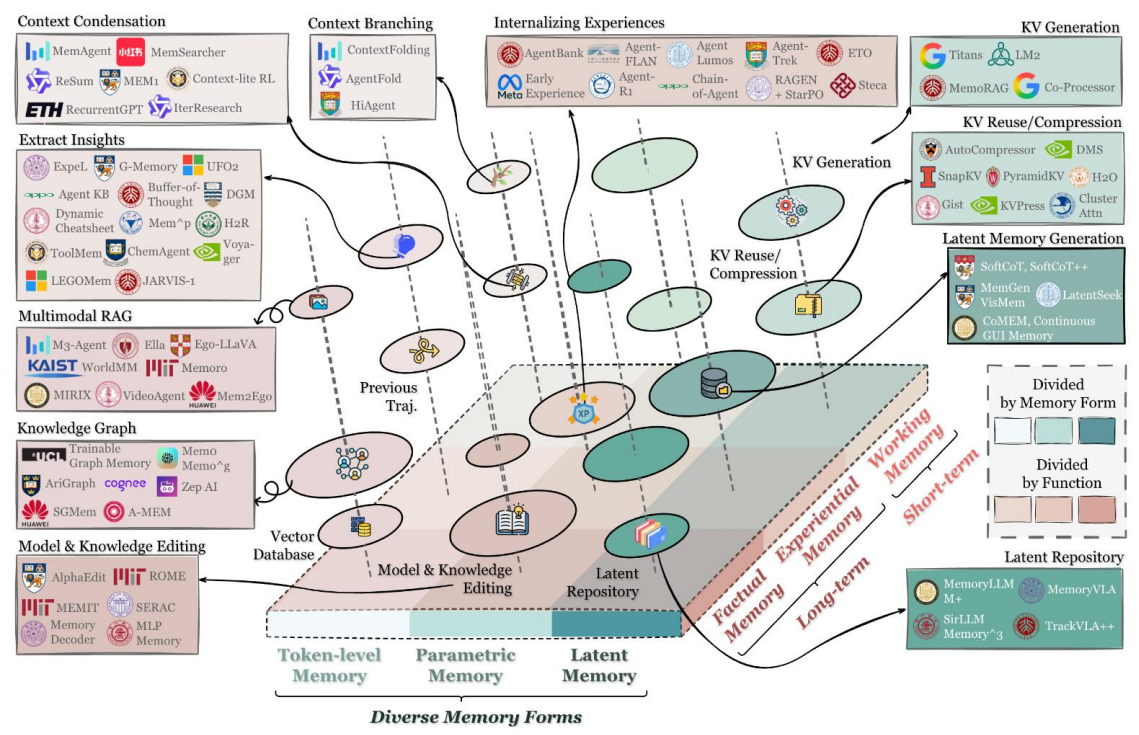
ReAct的执行过程可以总结为一个固定的 "三步循环"

04

Agent Memory

Agent Memory 是面向行动的显式长期状态管理

- Agent memory 是 agent 用来存储、组织、更新、检索和利用历史信息的机制，使其能在长期交互中保持连续性、个性化、经验复用和自我改进。
 - ✓ Forms: memory 以什么形态存在，决定可检索性、可解释性、更新方式和 planner 接口。
 - ✓ Functions: memory 存什么，决定 memory 是否真正改善 future decisions。
 - ✓ Dynamics: memory 如何变化，Dynamics 决定长期 agent 是否能保持准确、稳定和可治理。



先看记忆是什么形式，再看它做什么，最后看它如何随交互变化。



AVC Lab

Affective Vision Computing Lab

用Agent Memory 去解决具身中长任务的现实矛盾

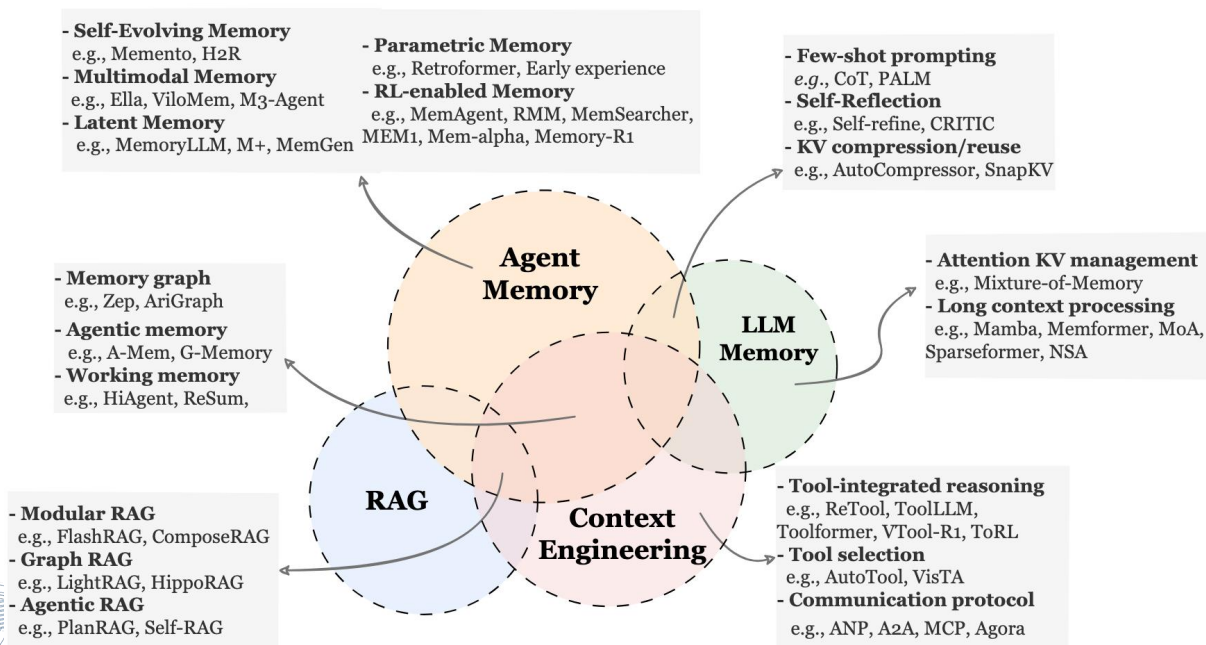
- 具身智能如果没有memory，一旦面临长任务规划，就会出现核心矛盾
- 例如：
 - 家务类的 VLA：做饭、收拾动作本身不难，难在“过程很长、状态很多、目标会变”。机器人如果不记得刚刚把抹布放哪、上一轮为什么失败、哪个抽屉已经打开过，很快会在房间里来回打转。
 - 长距离的 VLN：多阶段导航往往不是“走到目的地”这么简单，而是“先到 A，再绕过障碍，再去 B，途中遇到岔路还要回溯”。没有记忆，就只能靠当前视野做“贪心决策”。
 - 这也是当前研究人员最想解决的现实矛盾。
- 换三个角度看记忆系统：
 - 形态 Forms：记忆“长成什么样”；
 - 功能 Functions：记忆“到底用来做什么”；
 - 动态 Dynamics：记忆“怎么跑起来”。

因为具身任务的痛点，正好分别对应“存什么、怎么用、怎么跟环境一起变化”。



Agent Memory vs Context Engineering vs RAG vs LLM Memory

- Agent Memory不是简单的上下文窗口扩展:
把更多历史对话塞进 prompt, 只是“把过去搬进来”, 不等于你真的有一个能运行的记忆系统。
- Agent Memory也不等同于 RAG:
 - RAG 更偏“外部检索 + 生成”, 可以看作记忆系统的一种工具形态
 - Agent 记忆通常还要负责形成与更新, 而不仅是查询。
- Agent Memory不是“提示词工程”的同义词:
 - prompt 可以提醒模型“要安全”“要记住”
 - 真正的记忆系统要回答三个硬问题: 存什么、什么时候取、取出来怎么影响行动。



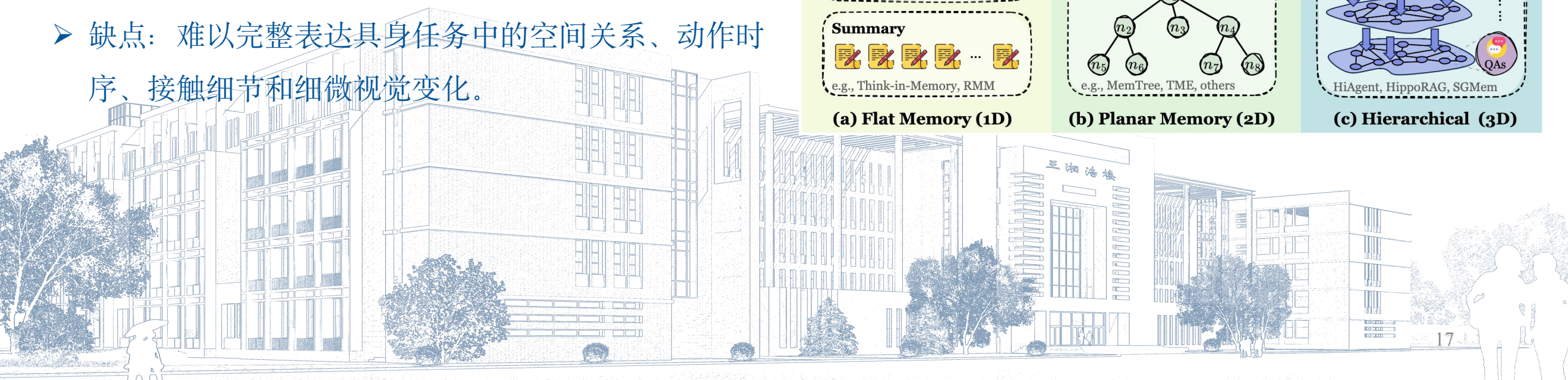
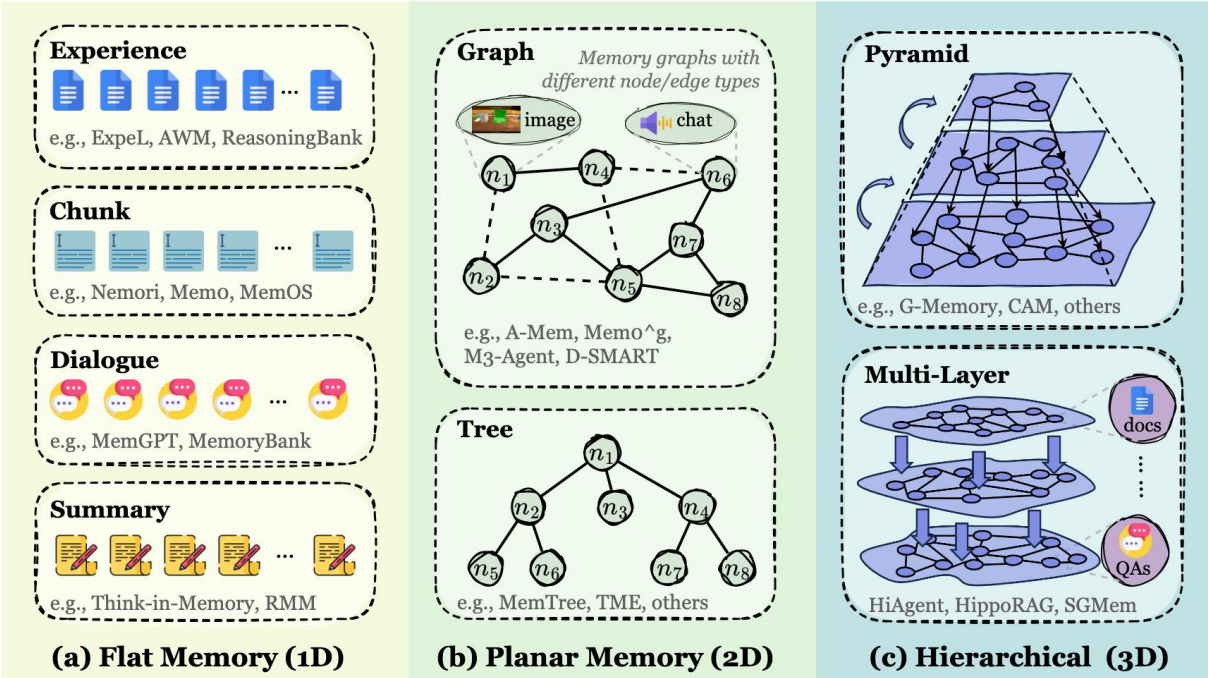
Form——Token-level memory



AVC Lab

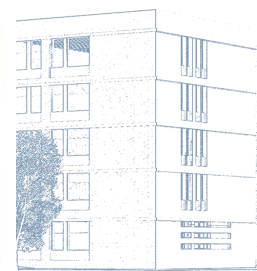
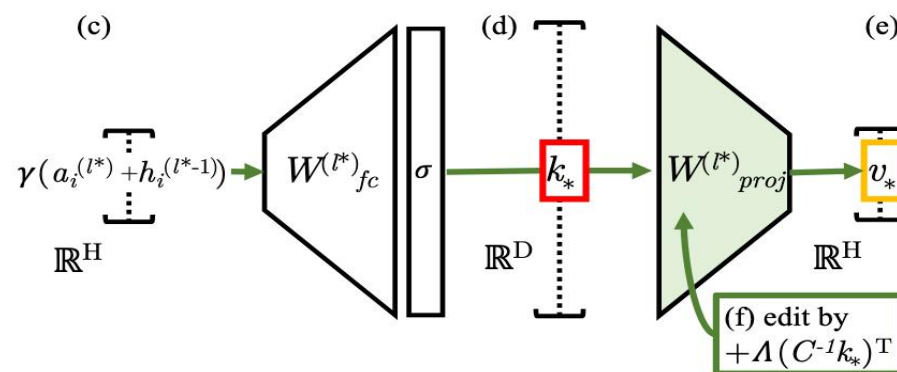
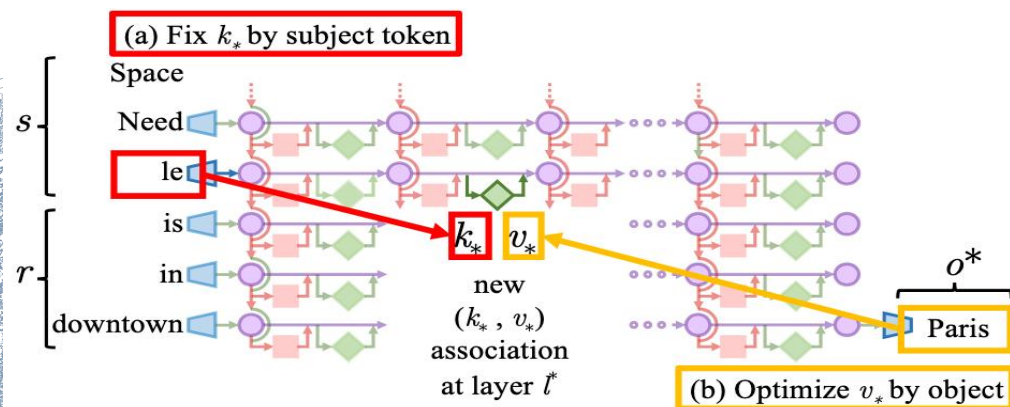
Affective Vision Computing Lab

- 定义：将经历编码为文本或结构化符号，并在需要时读取调用。
- 在具身系统对应“可解”的记忆：例如任务进度、子目标完成情况、曾经尝试过的策略摘要、失败原因的文字化描述等。
- 优点：人机皆可读，适合多阶段任务的计划与复盘。
- 缺点：难以完整表达具身任务中的空间关系、动作时序、接触细节和细微视觉变化。

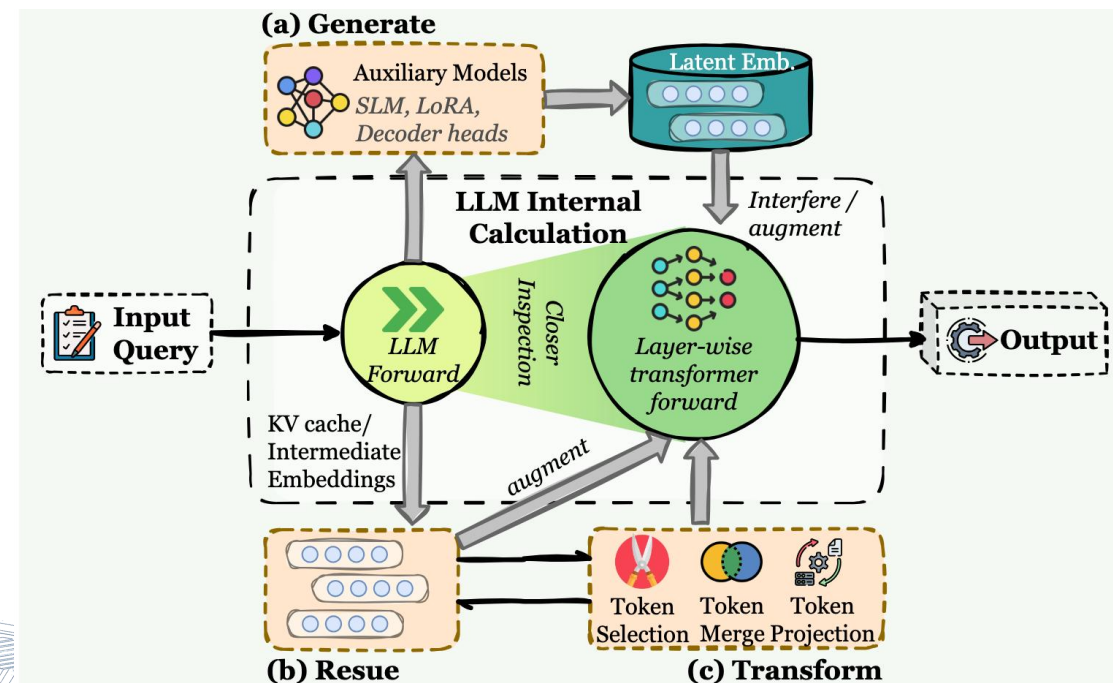


- 定义：通过训练或适配，将经验固化到模型参数中。
- 特点：无需外部存储，模型可直接调用所学规律。
- 对应具身智能的能力：通过反复训练形成稳定、熟练的策略。比如在某类房间布局、某类物体操作上逐渐形成“习惯性策略”，不需要每次都回忆具体案例。
- 优点：无需回忆具体案例，可直接调用经验。
- 缺点：更新慢、可控性弱，可能固化错误或噪声。
- 适用：适合沉淀长期技能，不适合存储临时状态。

Locating and Editing Factual Associations in GPT



- 定义：用向量或隐状态表示经历与世界。
- 机制：检索时做相似度匹配，更新时做向量融合或聚合。
- 对具身任务：优势：支持视觉、语言、动作、地图等多模态信息。
- 应用：适合 VLN 路线记忆、场景匹配和 VLA 经验检索。
- 缺点：在具身系统中的可解释性和可验证性弱，难以判断模型的思考依据与记忆内容，增加安全和调试难度。



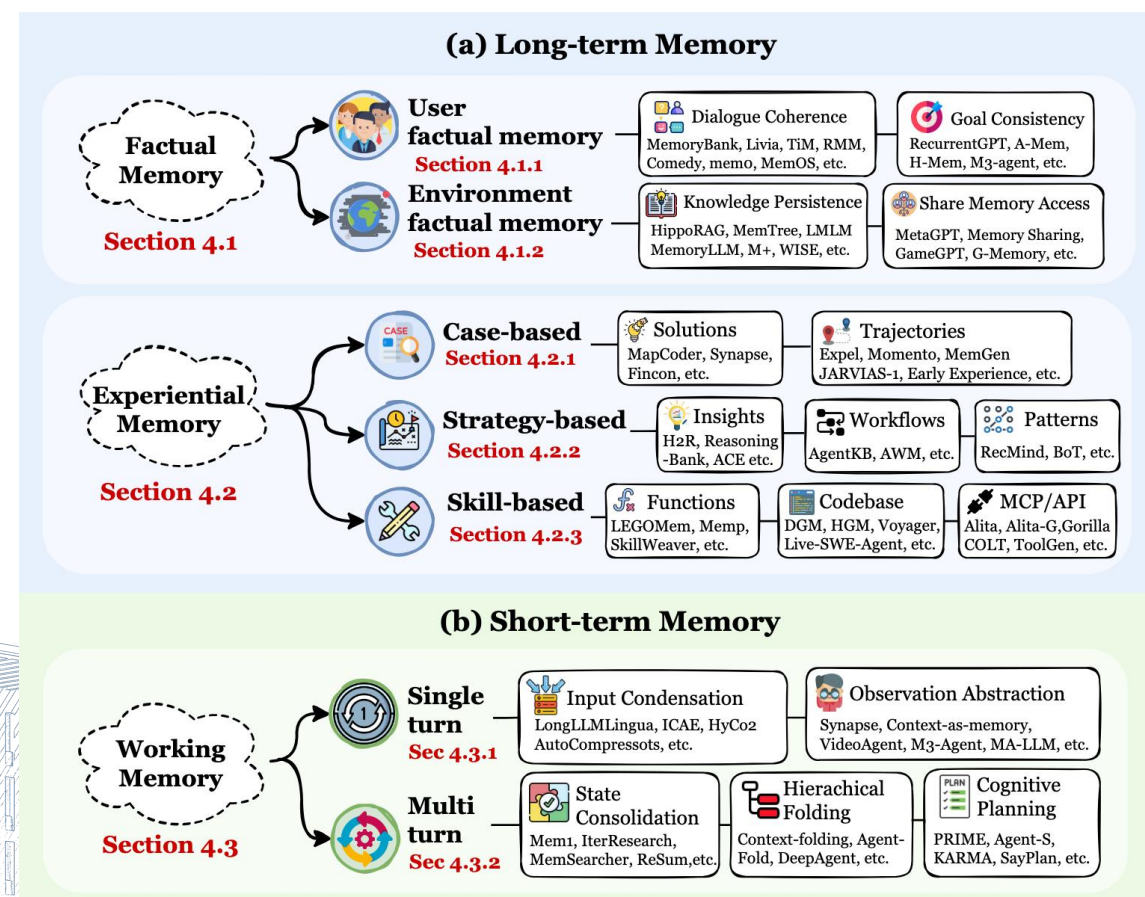
Functions



AVC Lab

Affective Vision Computing Lab

- 比“存在哪里”更重要的是“存来干什么”。
- 我们用三类功能来替代传统的长短期划分:
 - ✓ Factual memory
 - ✓ Experiential memory
 - ✓ Working memory



Functions——Factual memory

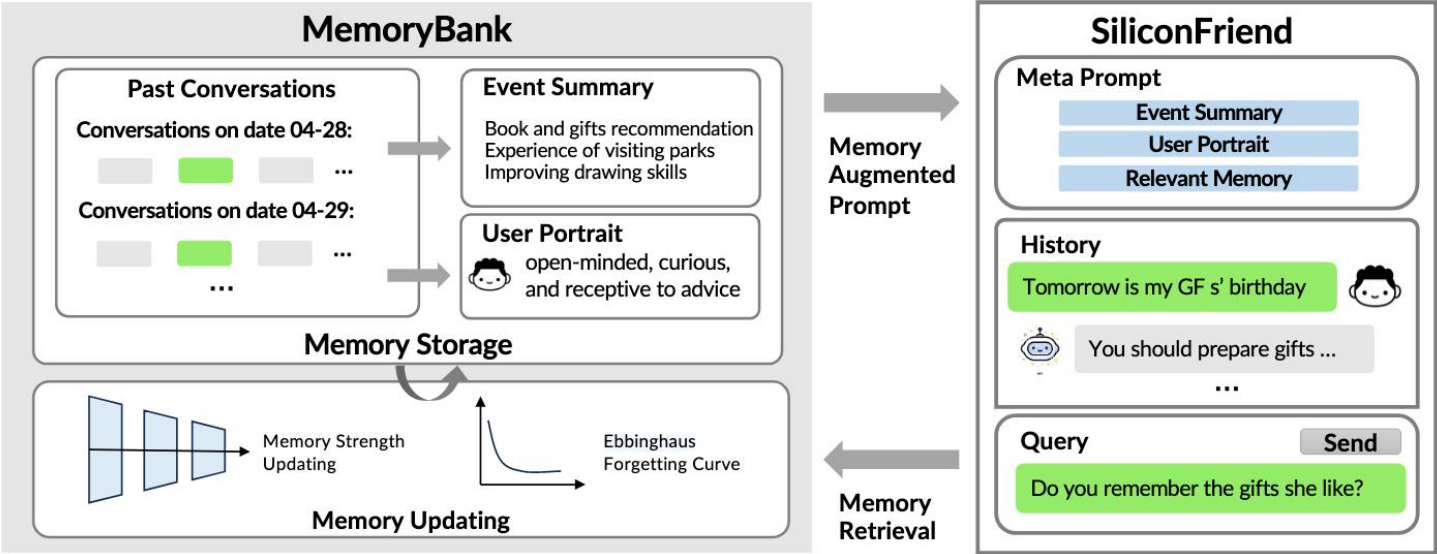


AVC Lab

Affective Vision Computing Lab

- 特点：记录客观环境信息，如物体位置、房间连通关系、危险区域和工具类别。
- 长期的、可声明的知识库，用来保证交互一致性、推理连贯性和适应性：
- VLN：记录路线与拓扑知识，如通行情况和场景连接关系。
- VLA：记录家务常识与环境事实，如物品位置和抽屉内容。
- 挑战：具身环境动态变化，事实可能过期；机器人需区分稳定事实与临时事实。

MemoryBank: Enhancing Large Language Models with Long-Term Memory (AAAI-2024)





Functions——Experiential memory

Case-based Memory

Store previous trajectories, solutions

e.g., Momento, Agent KB, JARVIS-1, ExpeL, MemGen, ...

Skill-based Memory

Distill previous trajectories into functions/APIs/MCPs/toolbox...

e.g., SkillWeaver, Voyager, Alita, Memp, Dynamic Cheatsheet, ...



Experiential Memory



Strategy-based Memory

Transform previous experience into strategies, templates or workflows

e.g., AWM, H2R, BrowserAgent, ACE, SE-Agents, ReasoningBank, ...

Hybrid Memory

Hybrid experiential memory

e.g., ExpeL, G-Memory, Memp, MemEvolve, Agent KB ...

- Experiential memory: 记住“我试过什么，结果如何”这是具身系统最实用、也最能提升“越跑越稳”的一类：把成功或失败的经历存下来。
- 对 VLN 来说：经验记忆常常对应“某条路线走不通”、“某个转角容易误判”、“某个语义线索经常是陷阱”。
对 VLA 来说：它能存“这类抓取姿态容易滑”、“这个物体轻拿会失败”、“这个接触顺序更稳定”。
- 可以把它理解成三层进化路径：① 从“记住一个具体案例”开始；② 逐步总结成“遇到这类情况就用这类策略”；③ 最后沉淀为“可复用技能”。这正好对应具身智能的发展节奏：从 demo 到可复用，从能跑到能稳。



Functions——Working memory

- Working memory更像“执行中的短期控制台”：当前子目标是什么，刚做了哪些动作，下一步要检查什么约束，当前观察是否与计划一致。
- 用于主动管理当前的临时上下文，支撑正在进行的任务执行。
 - 在多阶段 VLN 里，它是防止 agent “跑着跑着忘了自己要去哪里”的关键。
 - 在长程 VLA 家务里，它是让机器人“做完一步接下一步”的关键，否则就会出现反复开关柜门、重复拿放的行为。
- Working memory听起来像短期记忆，但关键点在于：它不是简单的缓存，而是一个能参与决策的结构化模块，和规划、执行紧密耦合。

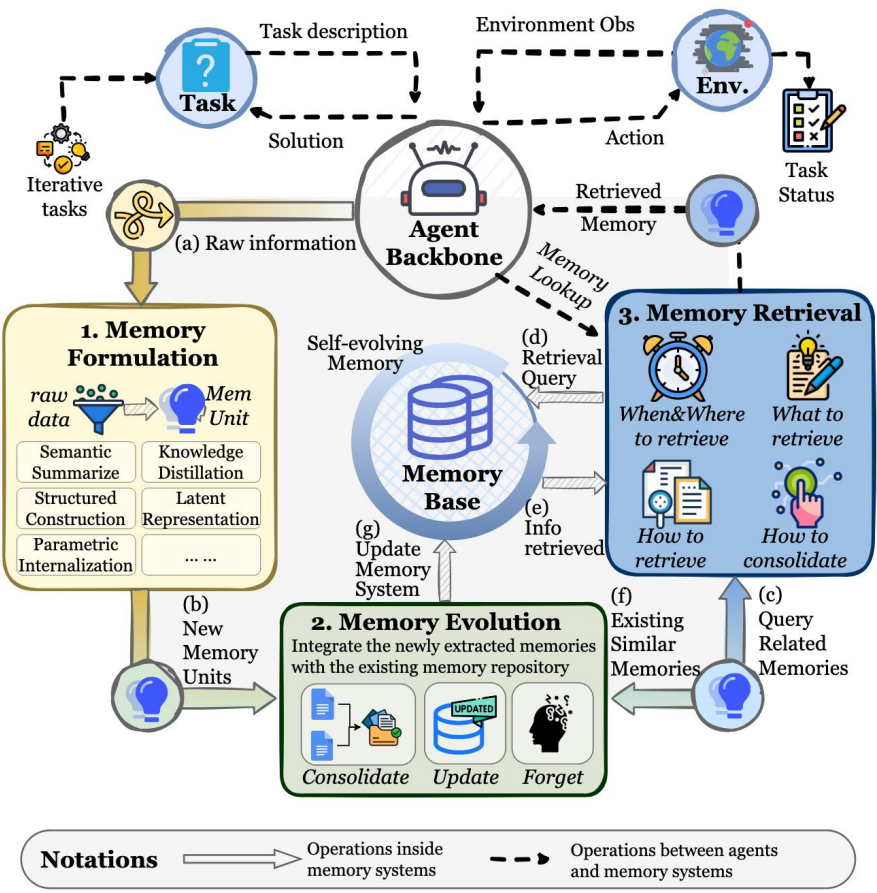
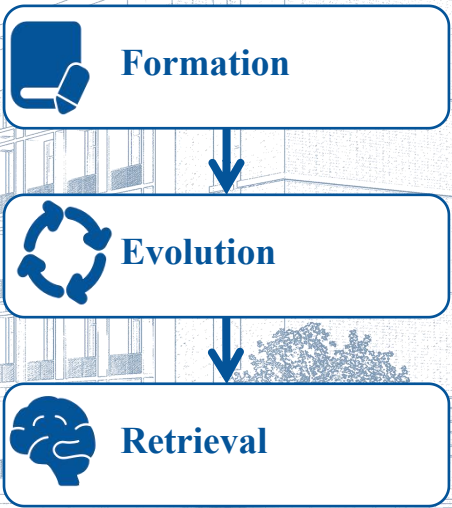


➤ 具身任务真正难点:

往往不在“有没有存储”，而在“记忆怎么跑起来”，这个就需要Dynamics。

➤ 把动态拆成三部分:

- ✓ formulation——记忆如何形成
- ✓ retrieval——记忆如何更新变化
- ✓ evolution——记忆如何被使用



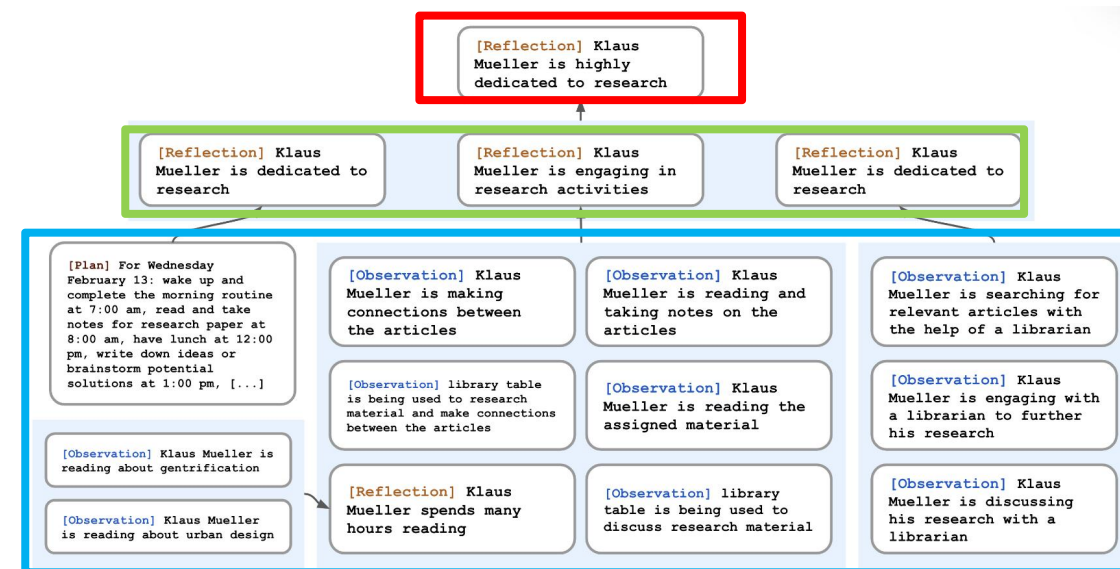
- formulation: 到底记什么, 不记什么?
- 具身系统的经历信息量巨大, 不可能全记。于是在形成阶段就要解决两个矛盾:

- ✓ 记忆过少, 关键失败原因丢失;
- ✓ 记忆过多, 检索成本爆炸, 影响实时性。

- 因此形成常常要做“压缩与结构化”:

- 把连续经历变成可检索条目,
- 过滤低价值噪声,
- 保留任务相关信号。

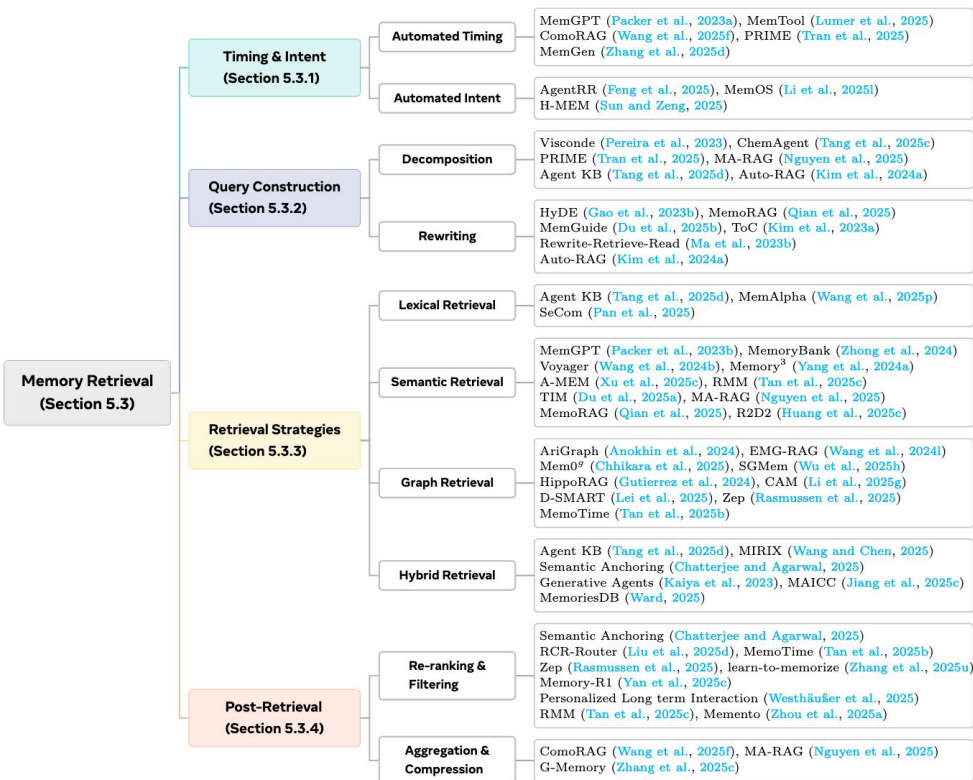
- 这在 VLA/VLN 里尤其关键, 因为长程任务面临的最大的难点就是“信息过载”。



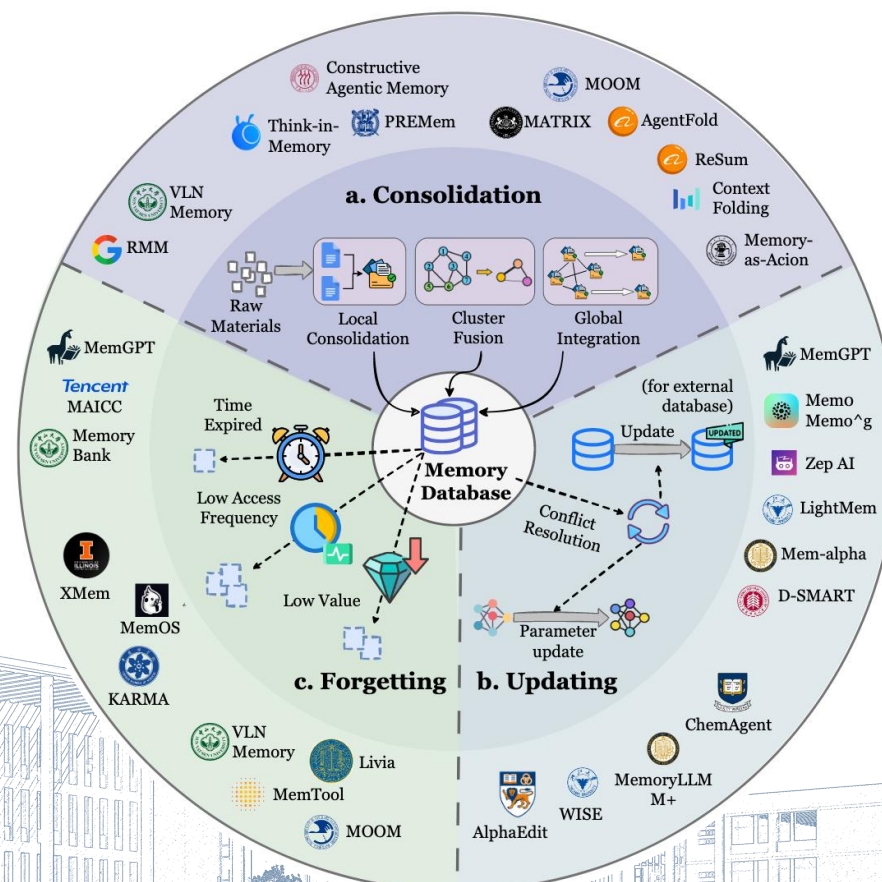
Generative Agents: Interactive Simulacra of Human Behavior (UIST '23 (ACM Symposium on User Interface Software and Technology)-2023)



- **retrieval**是记忆系统的灵魂。具身系统常见的问题不是“检索不到”，而是“检索到不该用的东西”：
 - ✓ VLN 里，场景看起来相似但其实路线不同，错用经验会走偏；
 - ✓ VLA 里，把旧的操作经验套到新物体上，可能导致不稳定接触或损坏。
- 因此**retrieval**不仅是相似度匹配，它还需要“任务对齐”：当前目标、当前阶段、当前约束必须参与判断，把新记忆整合进现有记忆库里，通过巩固、更新、遗忘保持知识库一致且高效，否则记忆会变成“联想过度”。



- 具身世界会变，机器人也会变。
- 一个成功的记忆系统必须支持**evolution**:
 - ✓ 新信息要能合并或覆盖旧信息；
 - ✓ 错误经验要能被纠正，而不是永久污染；
 - ✓ 低价值或过期信息要能被遗忘。
- 这点对长期部署的具身智能体非常关键：如果希望机器人“越跑越稳”，但前提是它的记忆不会越积越乱。



Generative Agents——25个AI在小镇生活，LLM记忆管理的开山之作

Generative Agents: Interactive Simulacra of Human Behavior (UIST '23 (ACM Symposium on User Interface Software and Technology)-2023)

Generative Agents 记忆架构：记忆流（存什么）→ 检索（怎么找）→ 反思（怎么提炼）。

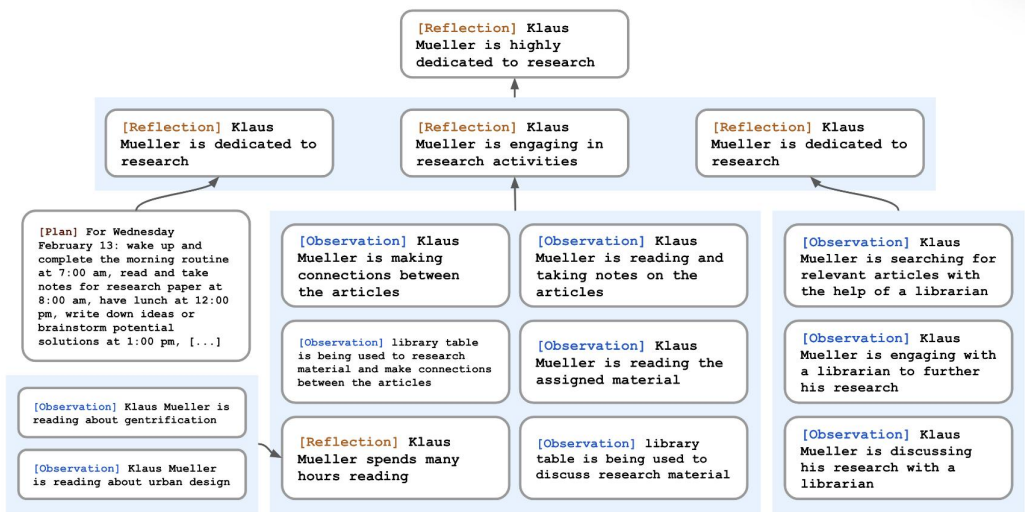
记忆流：用自然语言写日记每条记忆就是一句自然语言描述 + 时间戳；

三维检索：不是所有记忆都平等当AI需要决定下一步行动时，它要从记忆流中找到此时此刻最相关的记忆。

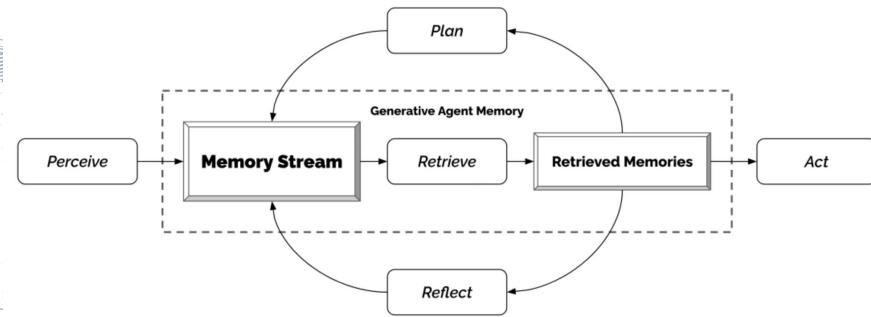
检索公式：得分 = 时效性 + 重要性 + 相关性（权重均为1）

反思：从流水账到抽象认知。反思的触发条件：当最近事件的重要性总分超过150时触发（约每天2-3次）。反思分三步。

- LLM提出3个高层次问题
- 以问题为query检索相关记忆
- LLM输出洞察，反思可嵌套



图：反思树——Klaus Mueller 的反思层级，从底层观察到高层自我认知



图：Generative Agents 架构图——记忆流 → 三维检索 → 反思 → 递归规划

MemGPT: 让AI像操作系统一样管理自己的记忆

MemGPT: Towards LLMs as Operating Systems (NeurIPS 2024)

把操作系统的虚拟内存分页机制搬进LLM——固定上下文窗口也能拥有"无限记忆"。

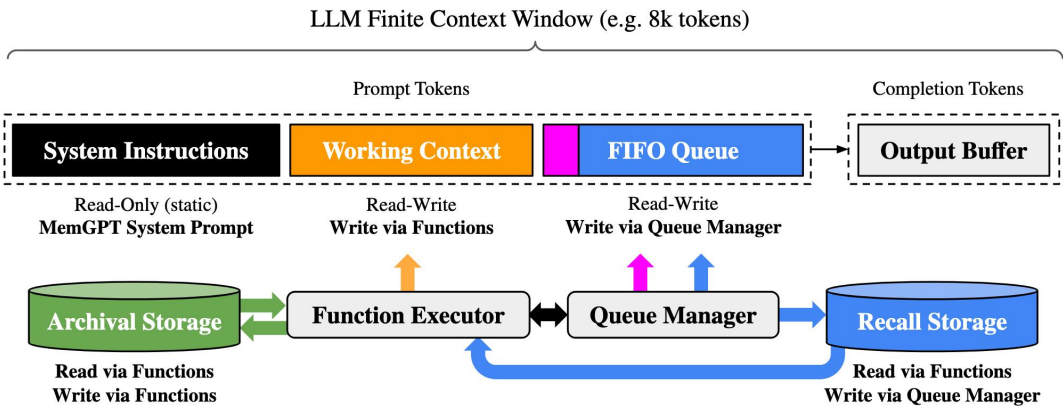
MemGPT系统流程——LLM处理器+主上下文+外部存储+函

数执行器的完整循环

驱逐机制:

- FIFO队列70%阈值→警告
- FIFO队列100%阈值→驱逐

LLM自主管理: MemGPT不给LLM预设检索策略, 而是提供一套函数接口, 让LLM自己决定什么时候做什么。



➤ 系统指令

➤ 工作上下文

➤ FIFO队列

MemoryBank用记忆曲线让大模型学会遗忘与铭记



AVC Lab

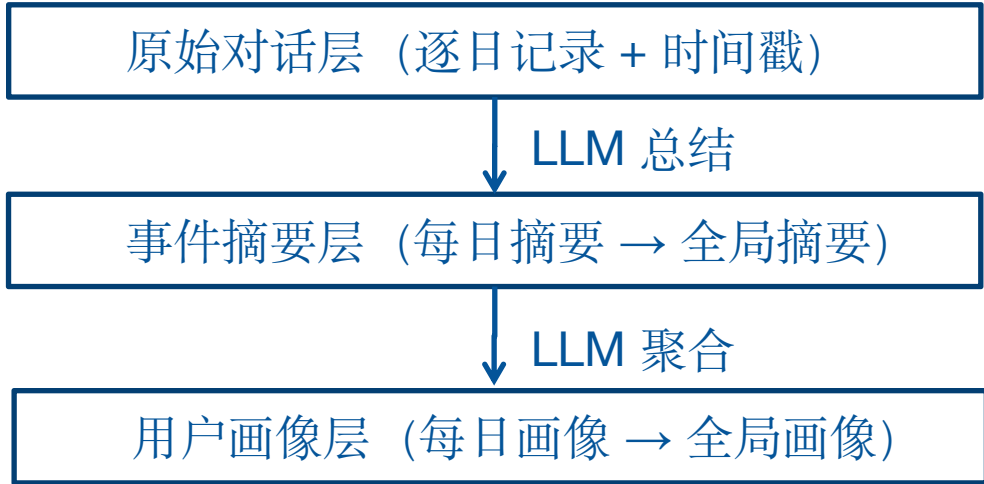
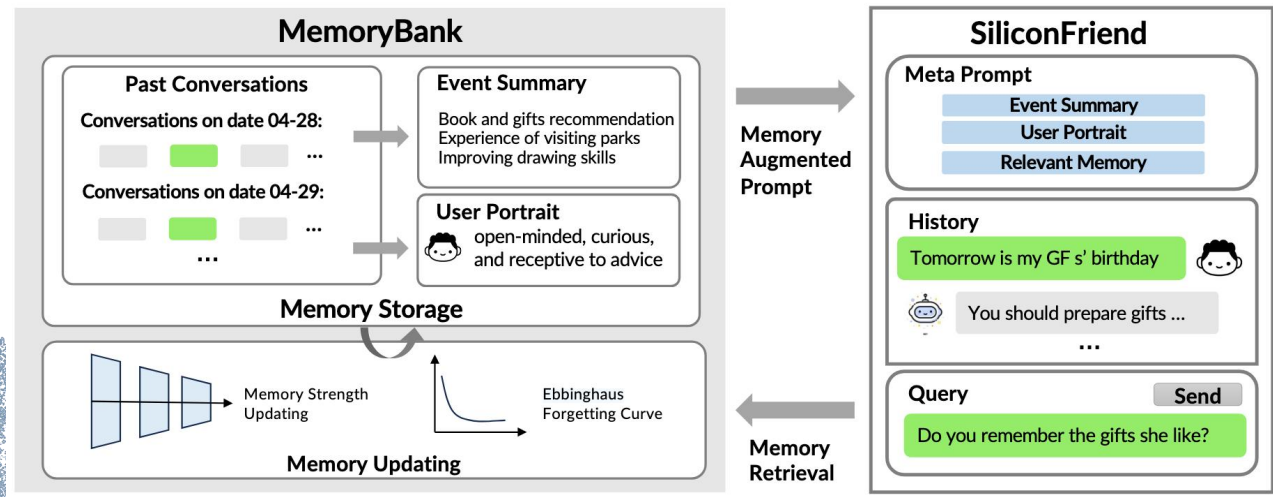
Affective Vision Computing Lab

MemoryBank: Enhancing Large Language Models with Long-Term Memory

用心理学中的艾宾浩斯遗忘曲线来驱动记忆更新 $R = e^{(-t/S)}$

MemoryBank 的核心洞察是：记忆管理可以脱离 LLM 的自主决策。

MemoryBank 把记忆更新建模为一个可预测的数学过程——用指数衰减模型计算每条记忆的保留强度，强度低于阈值就淘汰。



- 遗忘曲线只作用于第一层（原始对话层）中的每条记忆片段；
- 第二层（摘要层）和第三层（画像层）不受遗忘曲线影响

MemoryOS——AI Agent的"记忆操作系统"



AVC Lab

Affective Vision Computing Lab

Memory OS of AI Agent

能不能像操作系统管理内存那样，给 Agent 的记忆一个完整的管理系统？

三段分页存储 + Heat 驱动淘汰 + Persona 画像——MemoryOS 用可预测的算法（公式、阈值、FIFO）替代 LLM 的自主决策。记忆管理从"凭感觉"变成"有章程"。

MemoryOS 把记忆分三级：

- STM（短期记忆）——"对话页"队列。
- MTM（中期记忆）——同主题的对话页聚合成段（segment）。
- LPM（长期人格记忆）——最持久的"硬盘"

Heat 淘汰：哪段记忆值得留下？

$$Heat = \alpha \cdot N_{\text{visit}} + \beta \cdot L_{\text{interaction}} + \gamma \cdot R_{\text{recency}}, \quad (4)$$

Heat > τ （阈值 5）的段，触发 LPM 更新

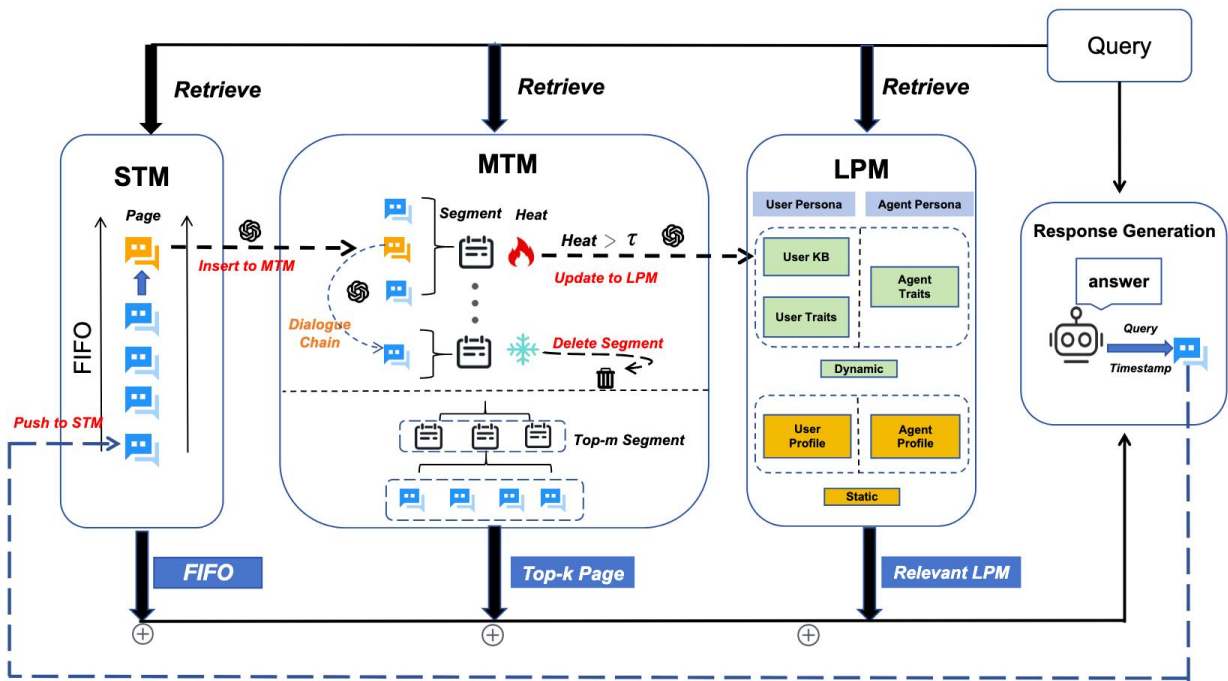


Figure 1: The overview architecture of MemoryOS, including memory Store, Updating, Retrieval, Response

Memory 设计流程：从观察写入到反思遗忘



设计阶段	关键问题	常用策略
需求分析	任务需要回忆事实、经历、偏好还是技能？	从 planning failure modes 反推 memory functions
写入策略	哪些信息值得进入长期记忆？	importance scoring、novelty filtering、privacy filter
组织方式	用 text/vector/schema/graph/skill library 哪种 form？	按 retrieval 和 update 需求选择，而不是默认向量化
检索与使用	当前 plan 需要哪些历史信息？	query rewriting、hybrid retrieval、reranking、planner injection
维护治理	错误、过期、冲突和敏感信息如何处理？	consolidation、conflict resolution、forgetting、audit log

设计 agent memory 时，应先确定任务需要哪种 function，再选择 form 和 dynamics，而不是先建一个向量库。



AVC Lab

Affective Vision Computing Lab

memory系统: Mem0、A-MEM 与 HiAgent 更强调可扩展和长时程

01

Mem0

面向 production-ready agents 的 long-term memory, 强调自动提取、更新、检索和可扩展部署。
工程化 memory。

02

A-MEM

强调 agentic / adaptive memory 管理: memory item 不是静态日志, 而会被组织、关联和演化。
动态记忆网络。

03

HiAgent

面向 long-horizon agent tasks 的 hierarchical working memory management, 把任务进度、子目标和上下文分层管理。

趋势: memory 系统正从存储模块走向自适应、可扩展的 agent 基础设施。

05

长程任务规划与Memory



Memory为什么会成为VLA/VLN的下一条主线

为什么“记忆”会成为 VLA /VLN 的下一条主战线？核心在于具身智能的长程交互需求无法仅靠大模型规模突破满足。

长程任务中的空间一致性、策略复用与连续执行，恰恰需要三类记忆协同支撑：

- 事实记忆让机器人知道“世界是什么样”，解决长期空间与物体信息的积累；
- 经验记忆让机器人知道“过去怎么做更稳”，解决失败复盘与策略沉淀；
- 工作记忆让机器人知道“此刻该干什么”，解决多阶段任务的连续执行。

三种形态也各司其职：

- ✓ token 帮助可解释与可控，
- ✓ latent 帮助多模态检索与相似关联，
- ✓ parametric 帮助长期技能固化。

这意味着，具身智能的竞争，本质就是记忆系统的竞争：谁能让智能体更高效地记、更灵活地用、更自主地更，谁就能先突破“易碎、低效、泛化差”的局限，真正走进物理世界。



没有 Memory, Planning 会丢失任务历史和世界状态

无 memory 的 planning 缺口

- 无法稳定记住原始 goal、用户约束和已经完成的 subgoals。
- 无法区分“未探索”“已探索但失败”“已成功完成”。
- 重复调用低效动作，或反复询问已经知道的信息。
- 失败后只做局部重试，缺少跨步骤诊断。

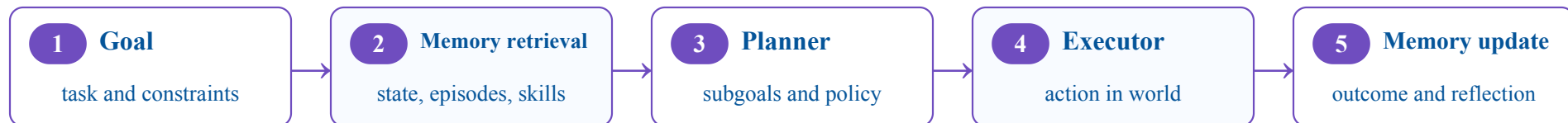
需要 memory 的原因

- Planning 需要 state: 当前世界、任务进度、资源和历史尝试。
- Planning 需要 priors: 哪些策略过去有效，哪些动作在当前对象上不可行。
- Planning 需要 continuity: 跨会话、跨 episode 延续目标与经验。
- Planning 需要 self-improvement: 把失败反思转化为未来策略。

Memory 提供 planning 所需的“过去”和“当前状态”，planning 决定 memory 何时被读写。



Planning-Memory Coupling: 二者形成闭环而不是并列模块



Memory -> Planning

- 提供 task state: 目标、进度、已完成和未完成条件。
- 提供 episodic priors: 过去相似任务中哪些步骤有效。
- 提供 procedural options: 可复用技能和工具调用方式。
- 提供 constraints: 用户偏好、安全规则和环境常识。

Planning -> Memory

- 计划决定当前需要检索什么，而不是全量召回历史。
- 执行结果决定哪些 observation 值得写入。
- 失败诊断会产生 reflection memory，影响未来 replanning。
- 完成的 subgoal 更新 task memory，避免 goal drift。

长时程具身 agent 的核心架构可以概括为：plan 决定行动路径，memory 维护历史状态，feedback 同时更新二者。

Voyager 展示了开放世界中的 skill memory 如何支撑长任务



AVC Lab

Affective Vision Computing Lab

Voyager: An Open-Ended Embodied Agent with Large Language Models

Minecraft 是一个典型的开放式环境 (Open-ended Environment)

- Voyager 在 Minecraft 中用 LLM 生成代码技能，并把成功技能写入 skill library。

自动课程

管理探索边界

迭代提示机制

建立执行反馈闭环



建立执行反馈闭环

将经验编译为程序

MEM: 多尺度具身记忆系统让机器人拥有"十五分钟记忆"

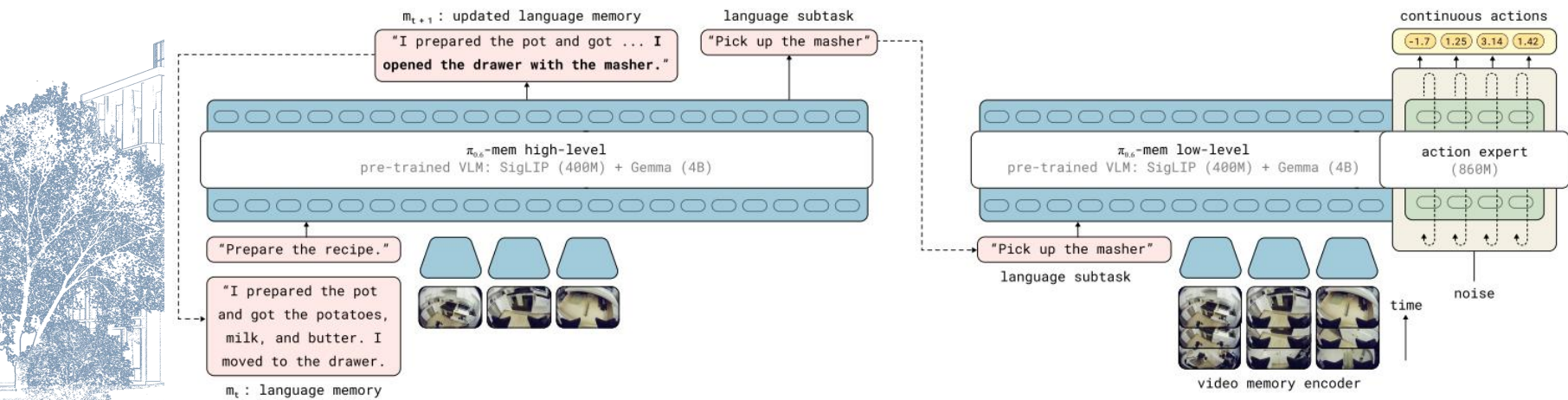
MEM (Multi-Scale Embodied Memory)

- 融合高效视频编码器与语言记忆机制，首次实现视觉-语言-动作模型在十五分钟时程上的有效记忆；
- 语言记忆是策略在解决任务过程中发生的语义事件概要。核心思想是训练高层策略根据前一时刻概要和当前观测及任务信息

m_t : I placed a plate in the cabinet and moved to the counter.

↓

m_{t+1} : I placed a plate in the cabinet, moved to the counter, and picked up a bowl.



06

Benchmark

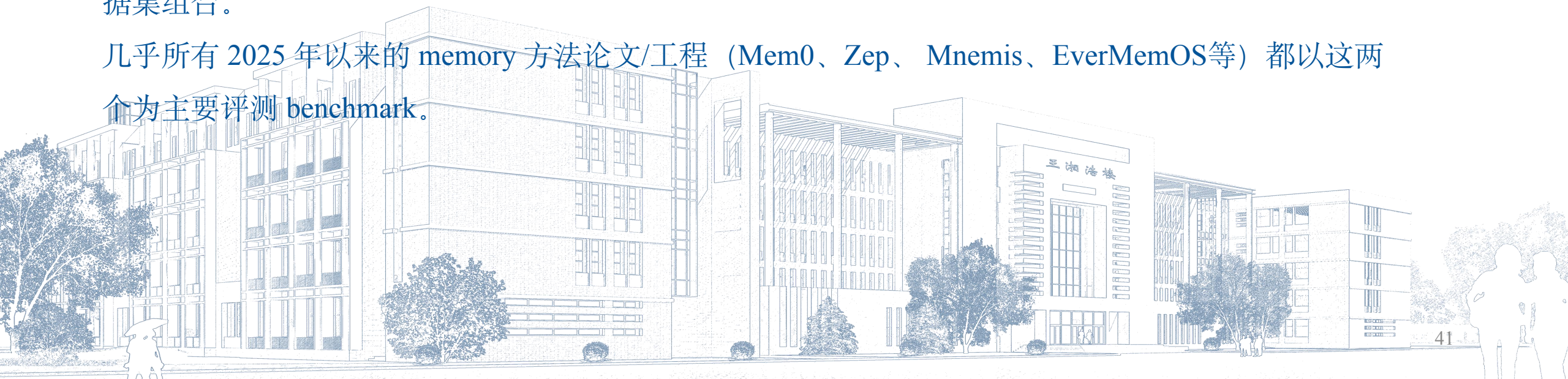
主要benchmark

目前关于评测记忆的benchmark很多，但是占据主导地位的还是LongMemEval和LoCoMo；学术界几乎形成了双标准的局面，绝大多数 2024–2026 的 memory 论文都同时在这两个上评测；

两个 benchmark 的使用对比

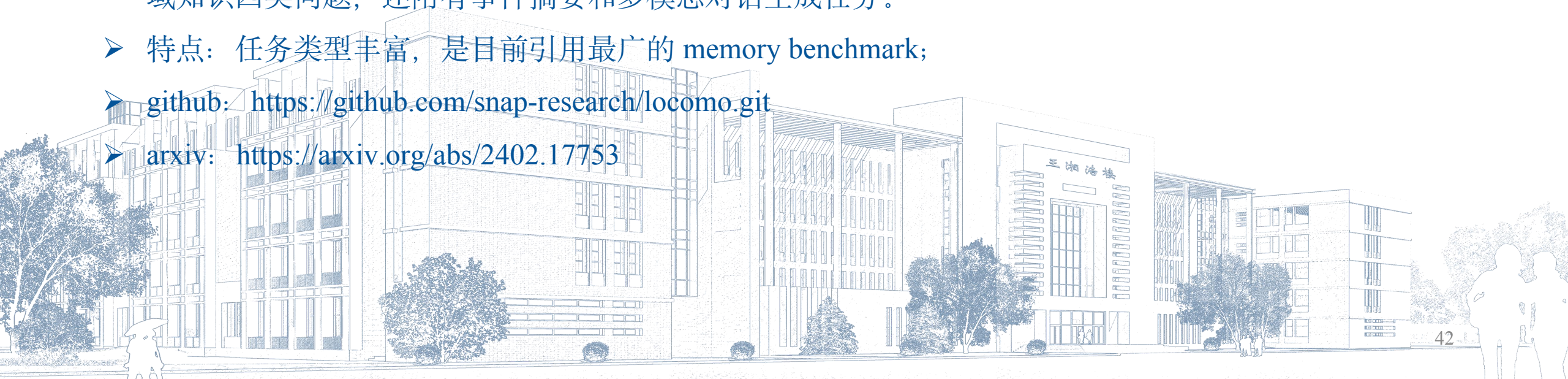
LoCoMo 数据集规模为 7,512 条，LongMemEval 为 500 条，两者是目前被同时使用最多的记忆评测数据集组合。

几乎所有 2025 年以来的 memory 方法论文/工程 (Mem0、Zep、Mnemis、EverMemOS等) 都以这两个为主要评测 benchmark。



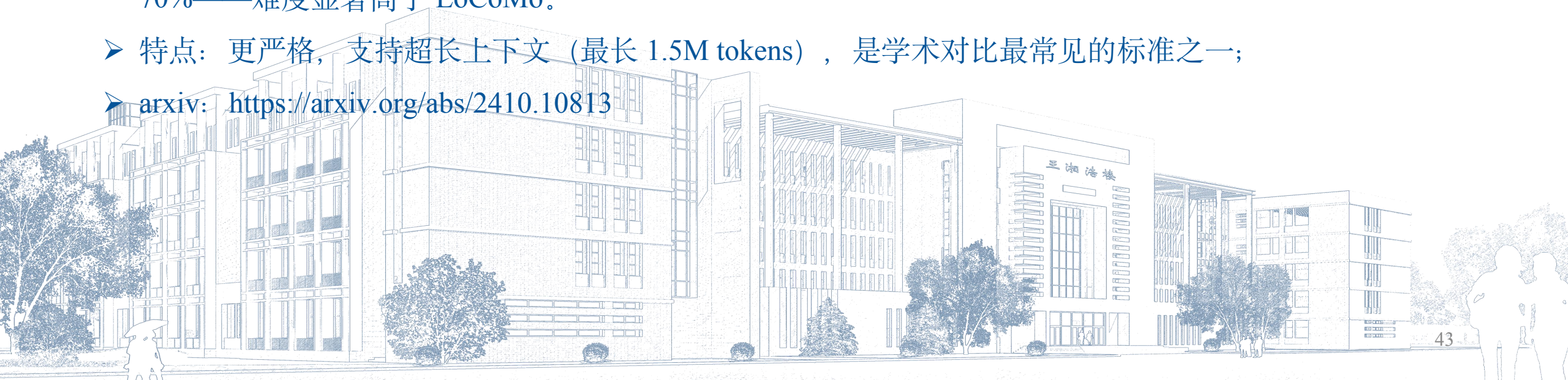
LoCoMo (ACL 2024)

- LoCoMo 用于评测 LLM agent 的超长期对话记忆，通过机器-人类混合流水线构建：两个 LLM agent 基于人物画像和时间事件图进行多轮对话，再由人工精修，每段对话平均跨越约 32 个 session、约 600 轮、~16K tokens。
- 数据：评测集包含 1,540 个 QA 样本，涵盖 单跳检索、多跳检索、时序推理、开放域知识四类问题，还附有事件摘要和多模态对话生成任务。
- 特点：任务类型丰富，是目前引用最广的 memory benchmark;
- github: <https://github.com/snap-research/locomo.git>
- arxiv: <https://arxiv.org/abs/2402.17753>



LongMemEval

- 数据: LongMemEval 包含 500 道人工标注题目, 测试五项核心记忆能力: 信息提取、多 session 推理、时序推理、知识更新、拒答判断。受 Needle-in-a-Haystack 范式启发, 设计了可控长度的对话历史流水线。
- 其标准版 LongMemEval-S 包含 500 段对话、每段约 115K tokens, 并模拟在线记忆处理场景。
- 初步评测发现, 长上下文 LLM 在该 benchmark 上性能下降 30% ~ 60%, 商业系统准确率仅 30% ~ 70%——难度显著高于 LoCoMo。
- 特点: 更严格, 支持超长上下文 (最长 1.5M tokens), 是学术对比最常见的标准之一;
- arxiv: <https://arxiv.org/abs/2410.10813>



目前的benchmark的困境

目前的benchmark评测“标尺”仍待校准，因为目前benchmark在评测记忆系统时，可能会被“刷分”，就会出现得分很高但在接入真实的业务场景时，却发现 Agent 依然频频失效：昨天说好的日程今天搞错时间、用户偏好被张冠李戴、甚至还会“脑补”出从未发生过的对话细节。

一场反讽实验

Supermemory.ai的初创公司发布了一篇最新博客，宣布在极度严苛的大模型长期记忆基准测试 (LongMemEval_S) 中，实现了惊人的99%准确率。他们采用的跑分手段如下：8变体“包抄”打法（准确率98.6%）：让8个特化Agent（如细节控、时间专家等）独立并行作答。

benchmark现在面临的不足

- 1、评判标准过于宽容: 行业通用的 LLM-as-Judge 基本逻辑是“相关即正确”，这种宽松的判定掩盖了当前记忆外挂的核心缺陷——召回冗余信息与过度生成；
- 2、考卷本身也存在错漏: 数据集中潜藏着部分逻辑颠倒、事实偏差的“脏数据”，干扰了评测精度。



吉林大学
JILIN UNIVERSITY

THANKS

